

1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari.

9-mashg’ulot. Pythonda ro‘yxatlarga doir masalalar yechish.

O‘quv savollari:

1. Pythonda ro‘yxatlarga doir oddiy masalalar yechish.
2. Ro‘yxatlarni ishlovchi metodlarga doir masalalar yechish.

1-savol: Talabalarning baholari

5 ta talabaning baholarini kiriting, ularni ro‘yxatga joylashtiring. So‘ngra, o‘rtacha bahoni hisoblang.

```
Na'muna:  
Kiritilgan baholar: 85, 92, 78, 90, 88  
Natija:  
O'rtacha baho: 86.6
```

Javob:

```
baholar = []  
for i in range(5):  
    baho = int(input(f"{i+1}-talabaning bahosini kiriting: "))  
    baholar.append(baho)  
  
avg = sum(baholar) / len(baholar)  
print(f"O'rtacha baho: {avg}")
```

2-savol: Talabalar va ularning yoshlarini kiritish

5 talabaning ismi va yoshini kiriting, ular uchun yangi ro‘yxat tuzing. Keyin faqat 18 yoshdan katta bo‘lgan talabalarni ro‘yxatga chiqarib bering.

```
Na'muna:  
Kiritilgan talabalar:  
Ism: Ali, Yosh: 20  
Ism: Vali, Yosh: 17  
Ism: Jasur, Yosh: 22  
Ism: Sami, Yosh: 16  
Ism: Zaynab, Yosh: 19  
Natija:  
18 yoshdan katta talabalar: [('Ali', 20), ('Jasur', 22), ('Zaynab', 19)]
```

Javob:

```
talabalar = []  
for i in range(5):  
    ism = input(f"{i+1}-talabaning ismini kiriting: ")  
    yosh = int(input(f"{ism}ning yoshini kiriting: "))  
    talabalar.append((ism, yosh))  
  
katta_yosh = [t for t in talabalar if t[1] > 18]  
print("18 yoshdan katta talabalar:", katta_yosh)
```

3-savol: Sonlarni kiritish va ularning eng kattasini topish

7 ta son kiriting va ulardan eng katta sonni toping.

```
Na'muna:  
Kiritilgan sonlar: 12, 7, 5, 64, 22, 11, 3  
Natija:  
Eng katta son: 64
```

Javob:

```
sonlar = []  
for i in range(7):  
    son = int(input(f"{i+1}-sonni kiriting: "))  
    sonlar.append(son)  
  
eng_katta = max(sonlar)  
print(f"Eng katta son: {eng_katta}")
```

4-savol: Ro'yxatdagi qiymatlarning tartibini tekshirish

5 ta son kiritib, ular **o'sish tartibida** berilganligini tekshirib chiqing. Agar berilgan tartibda bo'lsa, True chiqaring, aks holda False.

```
Na'muna:  
Kiritilgan sonlar: 3, 5, 12, 22, 64  
Natija:  
Tartib tekshiruvi: True
```

Javob:

```
sonlar = []  
for i in range(5):  
    son = int(input(f"{i+1}-sonni kiriting: "))  
    sonlar.append(son)  
  
tartib = sonlar == sorted(sonlar)  
print(f"Tartib tekshiruvi: {tartib}")
```

5-savol: Ro'yxatdagi sonlar orasidagi juft sonlarni filtrlash

6 ta son kiritilsin. Ro'yxatda faqat **juft sonlarni** ajrating va ularni yangi ro'yxatga joylashtiring.

```
Na'muna:  
Kiritilgan sonlar: 12, 7, 8, 5, 16, 3  
Natija:  
Juft sonlar: [12, 8, 16]
```

Javob:

```
sonlar = []  
for i in range(6):  
    son = int(input(f"{i+1}-sonni kiriting: "))
```

```
sonlar.append(son)
```

```
juft_sonlar = [x for x in sonlar if x % 2 == 0]  
print("Juft sonlar:", juft_sonlar)
```

6-savol: Ro'yxatdagi takroriy qiymatlarni aniqlash

7 ta mahsulot nomini kiriting. Takrorlanayotgan mahsulotlarni aniqlab, ularni ro'yxatga chiqarib bering.

```
Na'muna:  
Kiritilgan mahsulotlar:  
non, sut, olma, banan, sut, non, olma  
Natija:  
Takrorlanayotgan mahsulotlar: ['non', 'sut', 'olma']
```

Javob:

```
mahsulotlar = []  
for i in range(7):  
    mahsulot = input(f"{i+1}-mahsulot nomini kiriting: ")  
    mahsulotlar.append(mahsulot)  
  
takrorlanmalar = [x for x in set(mahsulotlar) if mahsulotlar.count(x) > 1]  
print("Takrorlanayotgan mahsulotlar:", takrorlanmalar)
```

Minimum va maksimum yig'indi



Beshta musbat butun son berilgan, ulardan to'rttasini ajratib olinganda umumiy yig'indisi bo'lishi mumkin bo'lgan minimum qiymat va maksimum qiymatni aniqlang.

Kiruvchi ma'lumotlar:

INPUT.TXT kirish faylining yagona satrida bo'sh joy bilan ajratilgan holda beshta butun son berilgan, sonlar `[1, 109]` oralig'iga tegishli.

Chiquvchi ma'lumotlar:

OUTPUT.TXT chiqish faylining yagona satrida bo'sh joy bilan ajratilgan holda avval minimum so'ng maksimum yig'indini chop eting!

Misollar

#	input.txt	output.txt
1	1 2 3 4 5 	10 14 

```
# Kiruvchi ma'lumotlarni o'qish
numbers = list(map(int, input().split()))

# Ro'yxatni tartiblash
numbers.sort()

# Minimal yig'indi (eng kichik yig'indini topish)
min_sum = sum(numbers[:-1])

# Maksimal yig'indi (eng katta yig'indini topish)
max_sum = sum(numbers[1:])

# Natijani chiqarish
print(min_sum, max_sum)
```

Yolg'iz son



Sizga butun sonlar to'plami berilgan. To'plamda 1 ta elementdan tashqari barchasini jufti bor. To'plamdagi yagona jufti bo'lmagan yolg'iz sonni toping.

Masalan: [1, 2, 3, 4, 3, 2, 1] to'plamida yolg'iz son 4 sonidir.

Kiruvchi ma'lumotlar:

INPUT.TXT kirish faylining birinchi satrida bitta butun $N(1 \leq N < 100)$ soni, to'plam elementlari soni kiritiladi, ikkinchi satrida bo'sh joy bilan ajratilgan holda N ta butun son, to'plam elementlari kiritiladi. to'plam elementlari qiymati $[0 \dots 100]$ oralig'ida

Chiquvchi ma'lumotlar:

OUTPUT.TXT chiqish faylida bitta butun son, to'plamdagi yolg'iz sonni chop eting!

Misollar

#	input.txt	output.txt
1	1 1	1
2	3 1 1 2	2
3	5 0 0 1 2 1	2

```
# Kirish faylidan ma'lumotlarni o'qish
N = int(input()) # N ta element
numbers = list(map(int, input().split())) # Sonlar to'plami

# Yolg'iz sonni topish uchun XOR operatorini ishlatamiz
result = 0
for num in numbers:
    result ^= num # XOR operatori

# Natijani chiqarish
print(result)
```

Yangi yil sovg'asi



Uchta opa-singil TATU da o'qishadi. Ular yangi yilga viloyatga o'z uylariga qaytishdan oldin onalari uchun sovg'a olishmoqchi. Ular olmoqchi bo'lgan sovg'aning narxi N so'm. Yo'l xarajatlaridan tashqari opa-singillarning to'ng'ichida A so'm, o'rtanchasida B so'm va kichigida C so'm ortiqcha pul bor. Ular onalari uchun olmoqchi bo'lgan sovg'ani ola olishadimi yoki yo'qligini aniqlang.

Kiruvchi ma'lumotlar:

Birinchi satrda bitta butun son, N soni sovg'aning narxi kiritiladi. Ikkinchi satrda esa 3 ta butun son, A, B, C sonlari, mos ravishda opa singillarning yo'l haqidan tashqari ortiqcha pullari miqdori kiritiladi.

$$0 \leq N, A, B, C \leq 10^9$$

Chiquvchi ma'lumotlar:

Opa - singillar onalariga sovg'ani ola olishsa "Yes" aks holda "No" so'zini chiqaring.

Misollar

#	input.txt	output.txt
1	120000 70000 40000 20000	Yes

```
# Kiruvchi ma'lumotlarni o'qish
N = int(input()) # Sovg'aning narxi
A, B, C = map(int, input().split()) # Opa-singillarning ortiqcha pullari

# Ular birgalikda qancha pulga ega bo'lishlarini hisoblaymiz
total_money = A + B + C

# Sovg'ani olish mumkinmi?
if total_money >= N:
    print("Yes")
else:
    print("No")
```

2-max



$n(2 \leq n \leq 100)$ ta elementdan iborat butun sonli massiv berilgan. Massivning ikkinchi eng katta elementini aniqlang.

Kiruvchi ma'lumotlar:

Birinchi satrda massiv elementlar soni n natural soni beriladi. Keyingi qatorda n ta nomanfiy butun son, massiv elementlari beriladi. Barcha kiruvchi ma'lumotlar qiymati 100 dan oshmaydi.

Chiquvchi ma'lumotlar:

Massivning ikkinchi eng katta elementini chiqaring.

Misollar

#	input.txt	output.txt
1	5 1 5 2 3 4	4
2	6 3 5 5 2 2 3	5

```
# Kiruvchi ma'lumotlarni o'qish
n = int(input()) # Massiv elementlari soni
arr = list(map(int, input().split())) # Massiv elementlari

# Massivni tartiblash
arr.sort()

# Ikkinchi eng katta element
print(arr[-2]) # Tartiblangan massivning ikkinchi eng katta elementi
```

Maosh



Kadrlar bo'limida ish haqqini so'mda oladigan 3 nafar xodim ishlaydi. Ulardan eng yuqori maosh oluvchining maoshi eng kam maosh oluvchidan qancha farq qilishini aniqlash talab etiladi.

Kiruvchi ma'lumotlar:

Kirish faylining yagona qatori bo'sh joy bilan ajratilgan barcha xodimlarning ish haqi hajmini o'z ichiga oladi. Har bir ish haqi 10^5 dan oshmaydigan natural sonidir.

Chiquvchi ma'lumotlar:

Chiqish faylida siz bitta butun sonni chiqarishingiz kerak - maksimal va eng kam ish haqi o'rtasidagi farq.

Misollar

#	input.txt	output.txt
1	963 487 847	476

```
# Kiruvchi ma'lumotlarni o'qish
salaries = list(map(int, input().split())) # Xodimlarning ish haqini ro'yxatga olish

# Eng yuqori va eng kam ish haqini topish
max_salary = max(salaries)
min_salary = min(salaries)

# Farqni chiqarish
print(max_salary - min_salary)
```


Knight game



Ali va Vali quyidagich o'yin o'ynashmoqda. $n \times n$ shaxmat doskasi mavjud ular navbat bilan doskaga bittadan o'tni bir birini ura olmaydigan qilib joylashtiradilar. Oxirgi bo'lib o'tni joylashtirgan o'yinchi o'yinda g'olib bo'ladi. Ikkala o'yinchi ham optimal o'ynagan taqdirda kim g'olib bo'lishini aniqlang. O'yinni Ali boshlab beradi.

Kiruvchi ma'lumotlar:

Kirish faylida 1-qatorda $T(1 \leq T \leq 1000)$ testlar soni. Keyingi T ta qatorda $n(1 \leq n \leq 10^4)$ soni kiritiladi.

Chiquvchi ma'lumotlar:

Chiqish faylida har bir test uchun alohida qatorda, agar Ali yutsa 0 aks holda 1 ni chop eting.

Misollar

#	input.txt	output.txt
1	2 2 1	1 0

```
# Testlar sonini o'qish
```

```
T = int(input())
```

```
# Har bir test uchun natijani aniqlash
```

```
for _ in range(T):
```

```
    n = int(input()) # n shaxmat doskasi o'lchami
```

```
    # Agar n juft bo'lsa, Vali yutadi (1), aks holda Ali yutadi (0)
```

```
    if n % 2 == 0:
```

```
        print(1) # Vali yutadi
```

```
    else:
```

```
        print(0) # Ali yutadi
```

Sovg'a



Oppog'oy va yetti gnom ertagini barcha eshitgan bo'lsa kerak. Yetti gnom oppog'oyning tug'ilgan kuniga unga sovg'a olmoqchi bo'lishibdi. Agar yetti gnomning birinchisida a_1 tanga, ikkinchisida a_2 tanga va h.k. yettinchi gnomda a_7 tanga puli bor bo'lsa hamda oppog'oy uchun olmoqchi bo'lgan sovg'a narxi S tanga turadigan bo'lsa, ularga yana qancha pul kerak bo'ladi.

Kiruvchi ma'lumotlar:

Birinchi qatorda yetti son gnomlarning har birida bor tangalar miqdori.

Ikkinchi qatorda olinishi kerak bo'lgan sovg'a narxi S .

Barcha sonlar 1000 dan oshmaydigan natural sonlar hisoblanadi.

Chiquvchi ma'lumotlar:

Sovg'ani sotib olish uchun yetti gnom uchun yana nechta tanga kerak?

Misollar

#	input.txt	output.txt
1	1 2 3 4 5 6 7 100	72
2	1 2 3 4 5 6 7 28	0

```
# Kiruvchi ma'lumotlarni o'qish
gnom_tangalar = list(map(int, input().split())) # Gnomlar tanga miqdori
S = int(input()) # Sovg'aning narxi

# Gnomlar yig'gan tangalar miqdori
total_tangalar = sum(gnom_tangalar)

# Sovg'ani sotib olish uchun kerak bo'lgan qo'shimcha tanga miqdori
if total_tangalar >= S:
    print(0) # Gnomlar to'plagan tangalar yetarli
else:
    print(S - total_tangalar) # Yetarli bo'lmagan miqdor
```

Imtihon daftarlari



Odatda yakuniy imtihonlari daftarda olinib, tekshirish uchun o'qituvchiga daftar yuzi olingan holda daftarning ichki qismi ketma-ket raqamlab beriladi.

O'tkirda bu safar Robo21 guruhning daftarni tekshirish topshirildi. Lekin O'tkir daftar raqamlarni ko'rib chiqqach kimdir daftarlarga teginganini sezib qoldi.

Buni aniqlash uchun O'tkirda yordam bering.

Kiruvchi ma'lumotlar:

Birinchi qatorda N talabalar soni $(1 \leq N \leq 100)$, ikkinchi qatorda 1 dan N gacha daftar raqamlari.

Chiquvchi ma'lumotlar:

Daftarga hech kim teginmagan bo'lsa "YES" aksi holda "NO" chiqaring.

Misollar

#	input.txt	output.txt
1	5 1 2 3 4 5	YES
2	10 9 4 3 2 5 7 6 8 1 10	NO

```
# Kiruvchi ma'lumotlarni o'qish
N = int(input()) # Talabalar soni
daftar_raqamlari = list(map(int, input().split())) # Daftar raqamlari

# Daftar raqamlari ketma-ket bo'lsa, ular 1 dan N gacha bo'lishi kerak
if daftar_raqamlari == list(range(1, N + 1)):
    print("YES")
else:
    print("NO")
```

Sumalak toshlari



Dasturchilar Klubi jamoasi har yili birgalikda sumalak tayyorlash uchun bir joyga yig'ilishadi. Bu yil ham sumalak tayyorlash ishlari avjida. Ammo sumalakka solinadigan M ta toshlar yo'q edi. N ta bola tosh keltirish uchun jo'nab ketishdi va har biri a_i ta tosh keltirdi. Sizning vazifangiz sumalakka M ta tosh solish uchun eng kamida nechta bolaning tergan toshlari tanlanadi?

Kiruvchi ma'lumotlar:

Birinci qatorda M va N natural sonlari. Ikkinchi qatorda esa mos ravishta N ta bolaning keltirgan a_i toshlari soni. ($1 \leq M, N, a_i \leq 1000$)

Chiquvchi ma'lumotlar:

Yagona qatorda sumalakka M ta tosh solish uchun kamida nechta boladan toshlar olinishini chiqaring. Agar toshlar yetarli bo'lmasa -1 chiqaring

Misollar

#	input.txt	output.txt
1	<pre>20 7 2 6 9 4 5 7 1</pre>	3

```
# Kiruvchi ma'lumotlarni o'qish
M, N = map(int, input().split()) # M toshlar soni, N bolalar soni
toshlar = list(map(int, input().split())) # Har bir bolaning olib kelgan toshlari

# Toshlar ro'yxatini kamayish tartibida saralash
toshlar.sort(reverse=True)

# Sumalakka kerakli toshlar sonini yig'ish
current_sum = 0
count = 0

for tosh in toshlar:
    current_sum += tosh
    count += 1
    if current_sum >= M:
        break

# Agar kerakli toshlar yig'ilsa, javobni chiqaramiz
if current_sum >= M:
    print(count)
else:
    print(-1)
```

Massivdan o'chirish



N ta elementdan iborat massiv berilgan. Massiv qiymatlari [1, 100] oralig'idagi qiymatlardan tashkil topgan. Massivning barcha elementi qiymati bir xil bo'lishi uchun eng kamida nechta element o'chirilishi kerakligini aniqlang.

Kiruvchi ma'lumotlar:

Kirish faylining dastlabki satrida bitta butun son, $N(1 \leq N \leq 100)$ soni kiritiladi. Ikkinchi satrda N ta butun son, massiv elementlari qiymati kiritiladi.

Chiquvchi ma'lumotlar:

Chiqish faylida yagona butun son, masala javobini chop eting!

Misollar

#	input.txt	output.txt
1	5 3 3 2 1 3	2

```
# Kiruvchi ma'lumotlarni o'qish
N = int(input()) # N soni
arr = list(map(int, input().split())) # Massiv elementlari

# Elementlarning chastotasini hisoblash uchun lug'at
count_dict = {}

# Massivdagi har bir element uchun uning chastotasini hisoblash
for num in arr:
    if num in count_dict:
        count_dict[num] += 1
    else:
        count_dict[num] = 1

# Eng ko'p uchraydigan elementni topish
max_count = max(count_dict.values())

# O'chirish kerak bo'lgan elementlar sonini hisoblash
result = N - max_count

# Natijani chiqarish
print(result)
```

TUIT CUP



TUIT CUP musobaqasida keyingi bosqichga o'tgan ishtirokchilarni aniqlash uchun quyidagicha chora o'ylab topishibdi.

“Agar qatnashchining bali, musobaqada k – o'rinni egallagan ishtirokchining balidan kam bo'lmasa, hamda u musbat bo'lsa, qatnashchi keying bosqichga o'tadi” – musobaqa qoidalaridan parcha.

Musobaqada jami $n(n \geq k)$ ta ishtirokchi qatnashdi. Sizga ular to'plagan ballar ma'lum. Keying bosqichga nechta qatnashchi o'tishini aniqlang.

Kiruvchi ma'lumotlar:

Birinci qatorda sizga n va k sonlari beriladi ($1 \leq k \leq n \leq 50$).

Keyingi qatorda sizga n ta son beriladi, a_i – bu i – o'rindagi ishtirokchi to'plagan bal ($0 \leq a_i \leq 100$).

Chiquvchi ma'lumotlar:

Keying bosqichga nechta qatnashchi o'tishini aniqlang.

Misollar

#	input.txt	output.txt
1	8 5 10 9 8 7 7 7 5 5	6
2	4 2 0 0 0 0	0

```
# Kiruvchi ma'lumotlarni o'qish
n, k = map(int, input().split()) # n ishtirokchilar soni, k o'rin
balls = list(map(int, input().split())) # Ishtirokchilarning ballari

# k-chi o'rindagi ballni topamiz
threshold = sorted(balls, reverse=True)[k-1] # k-chi o'rindagi ballni topamiz
(qisqacha)

# Keying bosqichga o'tgan ishtirokchilar sonini hisoblash
qualified_count = sum(1 for ball in balls if ball >= threshold and ball > 0)

# Natijani chiqarish
print(qualified_count)
```

Minimal va maksimal yig'indi 2



Sizga N va M sonlari beriladi, shundan so'ng N ta son beriladi. Sizing vazifangiz berilgan N ta son ichidan N-M tasi tanlanganda tanlangan sonlarning yig'indisi bo'lishi mumkin bo'lgan eng katta qiymat va eng kichik qiymat orasidagi farqni toping.

Kiruvchi ma'lumotlar:

Kirish faylining dastlabki satrida bitta butun son, $T(1 \leq T \leq 10)$ testlar soni kiritiladi.

Har bir testning birinchi satrida ikkita butun son, N va M sonlari kiritiladi ($1 \leq N \leq 1000, 0 \leq M \leq N$), ikkinchi satrida esa $[1, 1000]$ oralig'idagi N ta son, masala shartida aytilgan sonlar kiritiladi.

Chiquvchi ma'lumotlar:

Har bir test uchun alohida qatorda bitta butun son, masala javobini chop eting.

Misollar

#	input.txt	output.txt
1	<pre>1 5 1 1 2 3 4 5</pre>	<pre>4</pre>

```
# Kiruvchi ma'lumotlarni o'qish
T = int(input()) # Testlar soni

# Har bir test uchun ishlaymiz
for _ in range(T):
    N, M = map(int, input().split()) # N va M sonlari
    numbers = list(map(int, input().split())) # N ta son

    # Sonlarni tartiblaymiz
    numbers.sort()

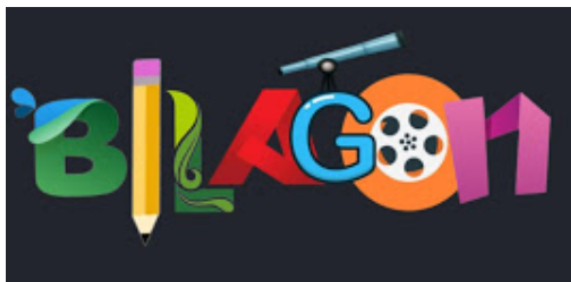
    # Eng kichik yig'indi: N-M ta eng kichik sonni tanlaymiz
    min_sum = sum(numbers[:N - M])

    # Eng katta yig'indi: N-M ta eng katta sonni tanlaymiz
    max_sum = sum(numbers[M:])

    # Farqni hisoblaymiz
    result = max_sum - min_sum

    # Natijani chiqarish
    print(result)
```

Kitobsevar BILAG'ON



Bilag'on kitob o'qishni juda ham yaxshi ko'radi, shuning uchun ham uning otasi har oylik ish maoshidan ma'lum bir qismini Bilag'onga kitoblar olish uchun sarflaydi. Bilag'onning otasi bu galgi oylik ish maoshidan Bilag'onga kitob olish uchun ko'pi bilan S so'mini sarflamoqchi. Bilag'onning otasi kitob do'koniga kirib qarasi u yerda faqat N ta kitob qolgan ekan, har bir kitobning narxi $A_i (1 \leq i \leq N)$ so'm ekanligi kitoblarning muqovasiga yopishtirib qo'yilgan. Bilag'onga qancha ko'p kitob sovg'a qilinsa shuncha ko'p xursand bo'lishini inobatga olgan holda Bilag'onning otasi imkoni boricha ko'p sondagi kitob olmoqchi, unga kitob uchun ajratgan S so'mi bilan ko'pi bilan nechta kitob olishi mumkinligini topishda yordam bering.

Kiruvchi ma'lumotlar:

Kirish faylining dastlabki satrida ikkita butun son, $N (1 \leq N \leq 10^5)$ va $S (1 \leq S \leq 10^9)$. Ikkinchi satrida bo'sh joy bilan ajratilgan holda N ta butun son, $A_i (1 \leq i \leq N, 1 \leq A_i \leq 10^9)$ – har bir kitobning narxi kiritiladi.

Chiquvchi ma'lumotlar:

Chiqish faylida yagona butun son, Bilag'onning otasi ko'pi bilan nechta kitob sotib olishi mumkinligini chop eting!

Misollar

#	input.txt	output.txt
1	4 7 1 2 3 4	3
2	5 15 3 7 2 9 4	3
3	7 50 1 12 5 111 200 1000 10	4

```
# Kiruvchi ma'lumotlarni o'qish
N, S = map(int, input().split()) # N kitoblar soni, S pul miqdori
book_prices = list(map(int, input().split())) # Kitob narxlari

# Kitob narxlarini o'sish tartibida saralash
book_prices.sort()

# Kitoblar sotib olishni hisoblash
total_cost = 0
count = 0

for price in book_prices:
    if total_cost + price <= S: # Agar kitobni olish mumkin bo'lsa
        total_cost += price
        count += 1
```



```
else:
```

```
    break # Pul tugadi, yana kitob olish mumkin emas
```

```
# Natijani chiqarish
```

```
print(count)
```

Kompyuter xonasida



TATU ning E blok 111-xonasi, kompyuter xonasidir. Va u yerda N ta kompyuter bir qator chiziqda joylashgan. Tez orada u xonada Shokirov Shodmon domlanning darsi boshlanadi. Shuning uchun ham hali xonaga kirmagan talabalar NB(darsda bo'lmadi) olmasligi uchun xonaga kirishni tezlashtirishlari lozim. Ammo aniqlandiki, bu xonadagi ba'zi kompyuterlardan foydalanib bo'lmaydi. Chunki u kompyuterlardan hozir kimdir foydalanyapti yoki ular buzuq. Xonaga kelgan har yangi talaba eshikka eng yaqin foydalanib bo'ladigan kompyuter oldiga o'tiradi va u kompyuterni yangi keladiganlar uchun foydalanilmaydigan qiladi. 1-kompyuter eshikka eng yaqin hisoblanadi.

Agar dars boshlanmasidan oldin xonaga K ta bola kirs, ular band qiladigan kompyuter o'rinlarini o'sish tartibida ekranga chiqaring. Foydalanib bo'ladigan kompyuterlar yetarlicha ekanligi kafolatlanadi.

Kiruvchi ma'lumotlar:

Birinchi qatorda ikkita butun son - N va $K(1 \leq K \leq N \leq 2 * 10^5)$ sonlari kiritiladi.

Keyingi qatorda N ta butun son kiritiladi. i -son: 1 bo'lsa, bu kompyuterdan foydalanib bo'lmaydigan, 0 bo'lsa esa bu kompyuterdan foydalanib bo'lishini anglatadi.

Chiquvchi ma'lumotlar:

Darsga ulgurgan talabalar o'tiradigan kompyuter o'rinlarini orqali chiqaring.

Misollar

#	input.txt	output.txt
1	7 3 1 0 1 1 0 0 1	2 5 6
2	3 2 0 0 0	1 2

```
from collections import deque
```

```
# Kiruvchi ma'lumotlarni o'qish
```

```
N, K = map(int, input().split()) # N kompyuterlar soni, K talabalar soni
computers = list(map(int, input().split())) # Kompyuterlar holati
```

```
# Bo'sh kompyuterlarni queue (navbat)ga qo'shamiz
available_computers = deque()
```

```
# Bo'sh kompyuterlarni ro'yxatga qo'shamiz
```

```
for i in range(N):
    if computers[i] == 0:
        available_computers.append(i + 1) # Kompyuterlar joylashuvi 1-indeksli
bo'lgani uchun +1 qo'shamiz
```

```
# Talabalar uchun kompyuterlar joylashuvi
```

```
seated_computers = []
```

```
# K ta talabani joylashtirish
```

```
for _ in range(K):
    if available_computers:
        seated_computers.append(available_computers.popleft()) # Birinchi bo'sh
kompyuterni olib, band qilamiz
```

```
# Natijani chiqarish  
print(*seated_computers)
```